

8-Queens Problem

1. Aim

To implement the **8-Queens problem** using an Artificial Intelligence search technique to place eight queens on an 8×8 chessboard such that no two queens attack each other.

2. Problem Statement

The 8-Queens problem is a classic Artificial Intelligence problem in which **eight queens must be placed on an 8×8 chessboard**. The queens must be positioned so that no two queens share the same row, column, or diagonal. The objective is to find a valid arrangement that satisfies all these constraints.

3. Solution

The problem can be solved using the **backtracking technique**. Queens are placed one row at a time, and each position is checked to ensure that the new queen does not conflict with any previously placed queen. If a safe position is not available, the algorithm backtracks to the previous row and tries another position. This process continues until all eight queens are successfully placed.

4. Algorithm

1. Start with an empty 8×8 chessboard.
2. Place a queen in the first row.

3. Check whether the selected position is safe.
4. A position is safe if no other queen exists in the same column or diagonal.
5. If the position is safe, place the queen and move to the next row.
6. If no safe position is available, backtrack to the previous row.
7. Move the previous queen to another possible position.
8. Repeat the process until all eight queens are placed.
9. Display the final chessboard arrangement.

5. Program

```
def is_safe(board, row, col):  
    for i in range(row):  
        # Same column  
        if board[i] == col:  
            return False  
  
        # Same diagonal  
        if abs(board[i] - col) == abs(i - row):  
            return False
```

```
return True
```

```
def solve_queens(board, row):
```

```
    if row == 8:
```

```
        return True
```

```
    for col in range(8):
```

```
        if is_safe(board, row, col):
```

```
            board[row] = col
```

```
            if solve_queens(board, row + 1):
```

```
                return True
```

```
        # Backtrack
```

```
        board[row] = -1
```

```
    return False
```

```
board = [-1] * 8
```

```
if solve_queens(board, 0):  
    print("Solution Found:\n")  
  
    for row in range(8):  
        for col in range(8):  
            if board[row] == col:  
                print("Q", end=" ")  
            else:  
                print(".", end=" ")  
        print()  
    else:  
        print("No solution exists.")
```

6. Result

The 8-Queens problem was successfully implemented using the **backtracking algorithm**. The program checks the possible positions of each queen and rejects positions that cause conflicts with previously placed queens.

Output:

Solution Found:

Q.....
Q...
Q
Q..
 ..Q.....
Q.
 .Q.....
 ...Q....

Thus, eight queens were successfully placed on the chessboard such that **no two queens attack each other**, satisfying the constraints of the 8-Queens problem.

The screenshot shows a Python IDE with a project named 'PythonProject10'. The file explorer on the left lists several files, including '8-Queen.py'. The main editor window displays the code for '8-Queen.py', which defines a function 'is_safe' to check if a queen can be placed at a given position on an 8x8 board. The code uses a list 'board' to represent the board, where 'board[i][j]' is 1 if a queen is at row 'i', column 'j', and 0 otherwise. The 'is_safe' function checks for conflicts in the same column, the same diagonal, and the same anti-diagonal. The main code block shows the initialization of the board and the search for a solution using a recursive function 'solve'.

```

1 N = 8
2 board = [[0] * N for _ in range(N)]
3
4 def is_safe(board, row, col):
5     # Check if there is a queen in the same column
6     for i in range(row):
7         if board[i][col] == 1:
8             return False
9
10    # Check if there is a queen in the same diagonal
11    i, j = row, col
12    while i >= 0 and j >= 0:
13        if board[i][j] == 1:
14            return False
15        i -= 1
16        j -= 1
17
18    # Check if there is a queen in the same anti-diagonal
19    i, j = row, col
20    while i <= 7 and j <= 7:
21        if board[i][j] == 1:
22            return False
23        i += 1
24        j += 1
25
26    return True
27
28 def solve(board, row):
29     if row == 8:
30         return True
31
32     for col in range(8):
33         if is_safe(board, row, col):
34             board[row][col] = 1
35             if solve(board, row + 1):
36                 return True
37             board[row][col] = 0
38
39     return False
40
41 solve(board, 0)
42
43 # Print the solution
44 for row in range(8):
45     for col in range(8):
46         print(board[row][col], end=" ")
47     print()
  
```

The Run console shows the output of the program, which is a list of 8-tuples representing the positions of the queens on the board. The first solution found is [1, 0, 0, 0, 0, 0, 0, 0], which corresponds to the first row of the chessboard shown in the image above. The console also shows the message 'Process finished with exit code 0'.